

École Militaire Polytechnique

Département Informatique

Rapport de Travaux Pratiques

Systèmes Distribués

Extension d'un Système de Messagerie Distribuée

Nom : BAKOUCHE
Prénom : Mahmoud
Groupe : RSD
Module : Systèmes Distribués
Année : 2025 – 2026

Mars 2026

Table des matières

1	Interfaces de Supervision des Serveurs de Messagerie	2
1.1	Objectif	2
1.2	Interface du serveur SMTP	2
1.3	Interface du serveur POP3	5
1.4	Interface du serveur IMAP	7
2	Serveur d'Authentification avec Java RMI	9
2.1	Objectif	9
2.2	Partie 1 – Démarrage du serveur RMI et client d'administration	9
2.3	Partie 2 – Gestion des comptes utilisateurs	11
2.4	Partie 3 – Intégration avec le serveur POP3	12
2.5	Partie 4 – Tests d'authentification valide et invalide	13
2.6	Partie 5 – Suppression d'un compte utilisateur	14
2.7	Partie 6 – Modification du mot de passe	15
2.8	Partie 7 – Validation complète côté serveur POP3	16
	Conclusion	18

Partie 1 – Interfaces de Supervision des Serveurs de Messagerie

1.1 Objectif

Cette partie consiste à développer des interfaces graphiques indépendantes pour superviser et contrôler les serveurs SMTP, POP3 et IMAP. Chaque interface permet de démarrer et d'arrêter le serveur correspondant, d'afficher en temps réel l'historique des commandes échangées avec les clients, et d'indiquer le nombre de clients connectés. Ces interfaces sont développées de manière autonome, sans recourir à un service d'authentification externe.

1.2 Interface du serveur SMTP

L'interface d'administration SMTP permet de contrôler le serveur sur le port 25 et d'observer les échanges avec les clients en temps réel.

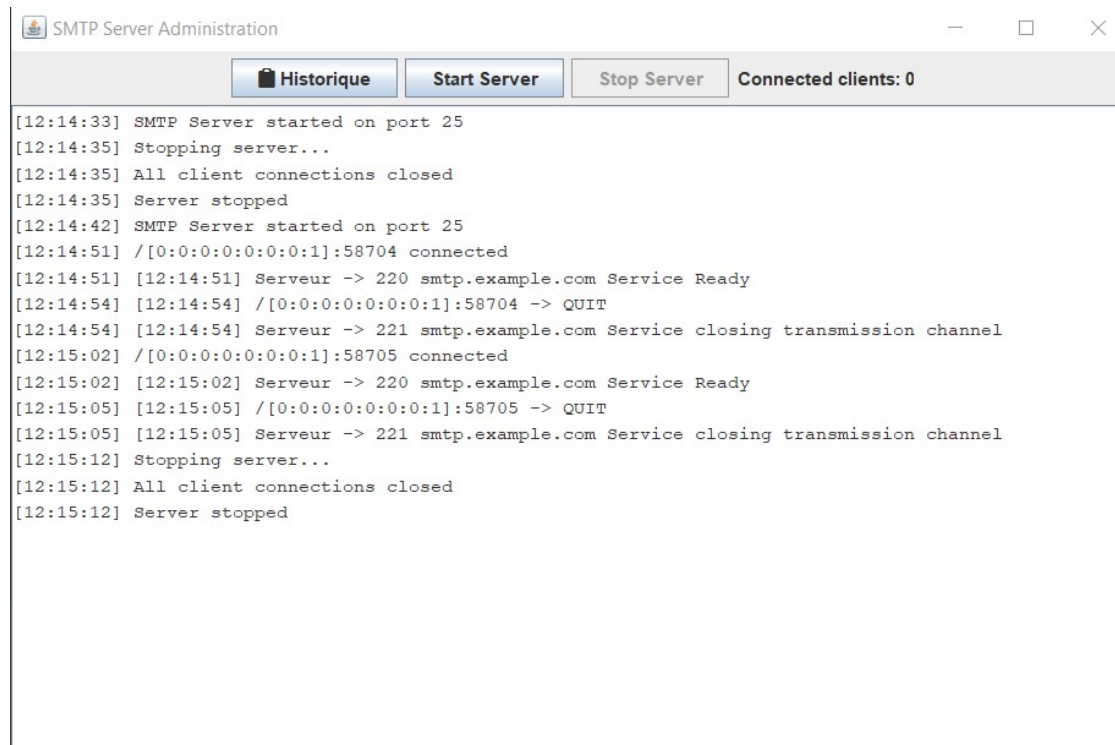
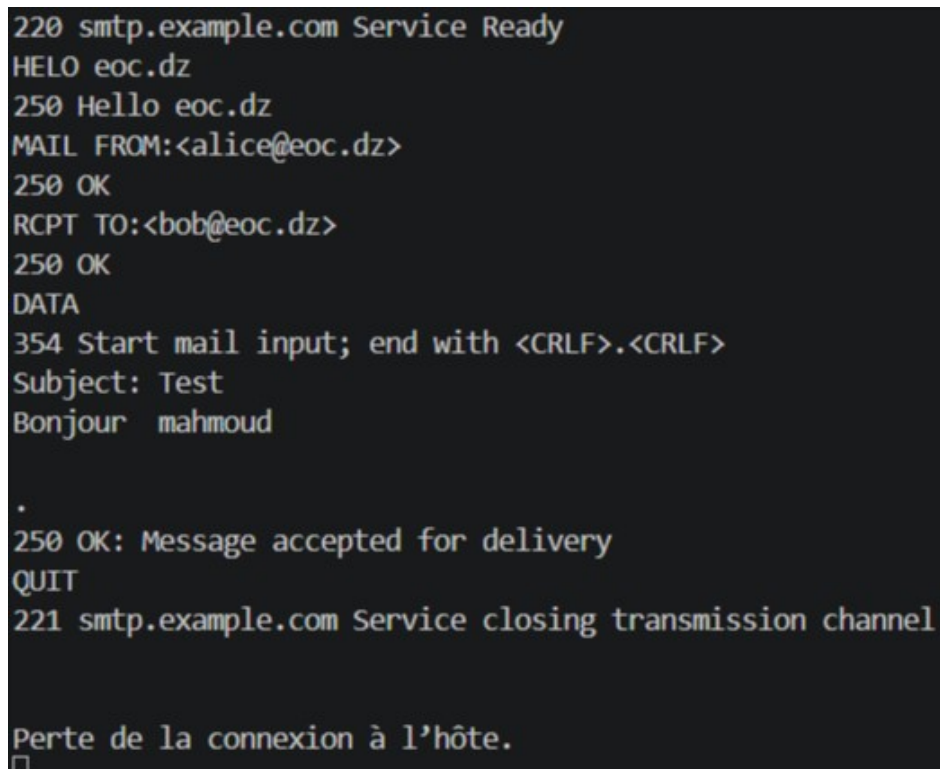


FIGURE 1.1 – Interface SMTP – démarrage et arrêt du serveur, historique des connexions clients

On observe sur cette capture que le serveur SMTP démarre correctement sur le port 25. Deux clients se connectent successivement, reçoivent la bannière 220 `smtp.example.com Service Ready`, envoient la commande `QUIT` et se déconnectent proprement. Le serveur est ensuite arrêté via le bouton *Stop Server*, ce qui déclenche la fermeture de toutes les connexions actives (*All client connections closed*).



```
220 smtp.example.com Service Ready
HELO eoc.dz
250 Hello eoc.dz
MAIL FROM:<alice@eoc.dz>
250 OK
RCPT TO:<bob@eoc.dz>
250 OK
DATA
354 Start mail input; end with <CRLF>.<CRLF>
Subject: Test
Bonjour mahmoud
.
250 OK: Message accepted for delivery
QUIT
221 smtp.example.com Service closing transmission channel

Perte de la connexion à l'hôte.
□
```

FIGURE 1.2 – Test SMTP via Telnet – envoi d’un message complet entre `alice@eoc.dz` et `bob@eoc.dz`

Cette capture montre un scénario d’envoi complet depuis Telnet. L’utilisateur `alice@eoc.dz` envoie un message à `bob@eoc.dz` avec le sujet *Test* et le corps *Bonjour mahmoud*. Le serveur répond `250 OK: Message accepted for delivery`, confirmant la bonne réception du message, puis ferme la session avec le code 221.

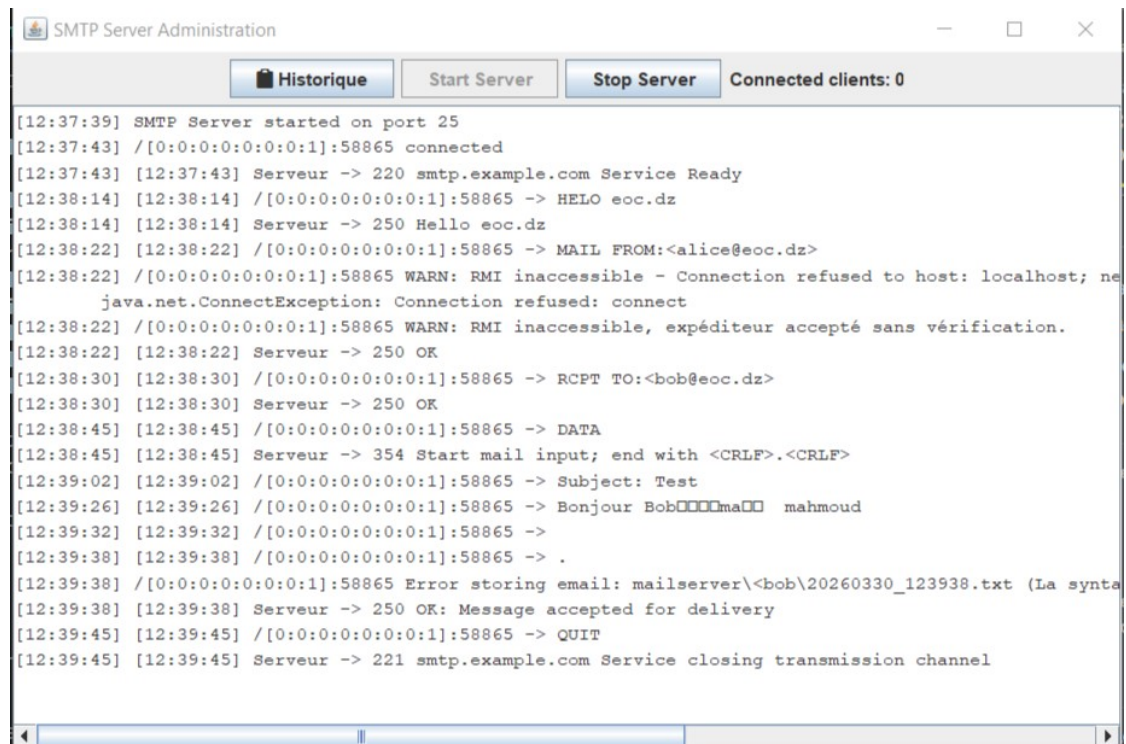


FIGURE 1.3 – Interface SMTP – historique complet d’une session HELO, MAIL FROM, RCPT TO, DATA, QUIT

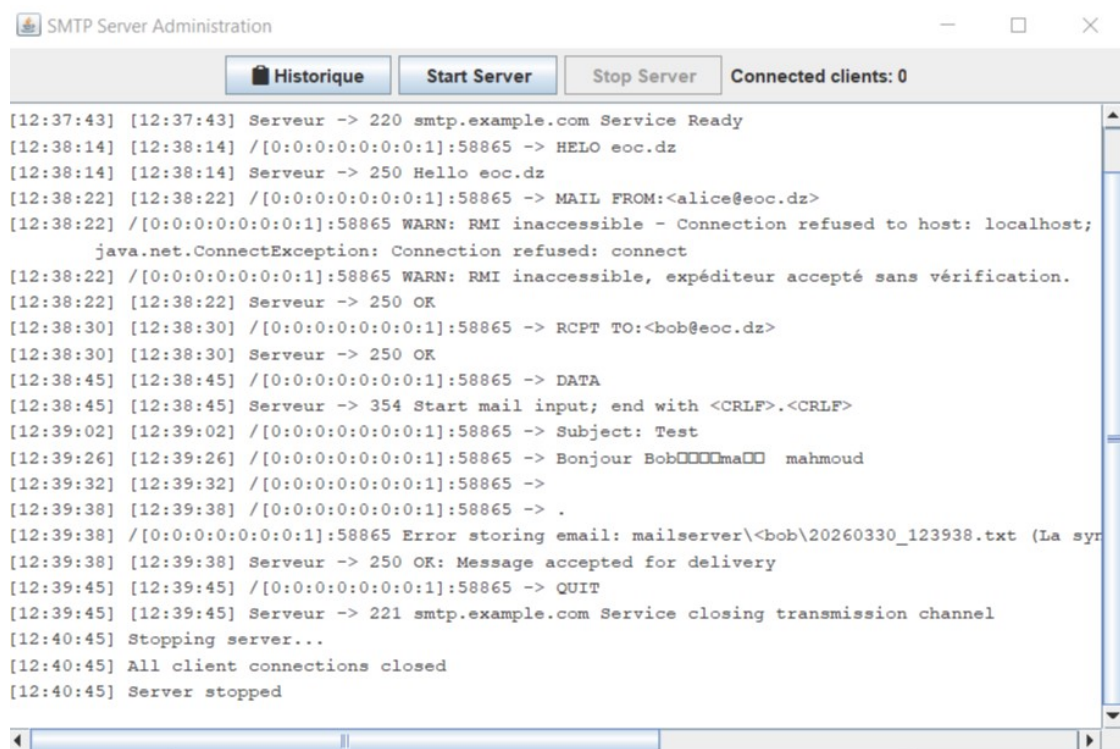


FIGURE 1.4 – Interface SMTP – arrêt propre du serveur (Stopping server... / Server stopped)

Les captures 3 et 4 illustrent une session d’envoi complète visible depuis l’interface d’administration. On peut suivre pas à pas les commandes HELO, MAIL FROM, RCPT TO, DATA et QUIT avec leurs réponses respectives. Une erreur de stockage du fichier message est visible (*Error storing email*) liée à un problème de chemin, mais le serveur répond tout de même 250 OK au client. La figure 4 montre l’arrêt propre du serveur à 12:40:45 avec la séquence *Stopping server... / All client connections closed / Server stopped*.

Analyse : Le serveur SMTP implémente correctement les codes de réponse standard (220, 250, 354, 221). L’interface offre une bonne visibilité sur les échanges. Une amélioration consisterait à corriger le chemin de stockage des messages et à distinguer visuellement les erreurs internes des refus protocole.

1.3 Interface du serveur POP3

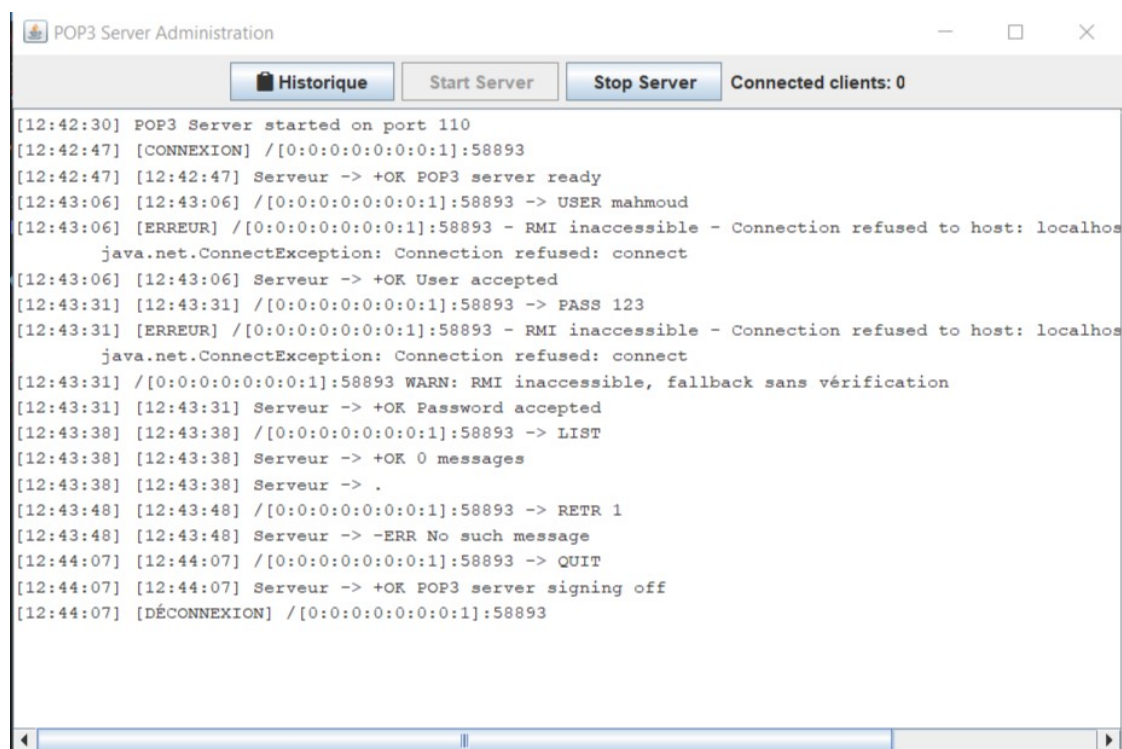


FIGURE 1.5 – Interface POP3 – session complète : connexion, USER, PASS, LIST, RETR, QUIT

```

Historique des Commandes — POP3

----- Nouvelle connexion : /[0:0:0:0:0:0:1]:58893 -----
[12:42:47] Serveur -> +OK POP3 server ready
[12:43:06] /[0:0:0:0:0:0:1]:58893 -> USER mahmoud
[ERREUR] /[0:0:0:0:0:0:1]:58893 - RMI inaccessible - Connection refused to host:
localhost; nested exception is:
    java.net.ConnectException: Connection refused: connect
[12:43:06] Serveur -> +OK User accepted
[12:43:31] /[0:0:0:0:0:0:1]:58893 -> PASS 123
[ERREUR] /[0:0:0:0:0:0:1]:58893 - RMI inaccessible - Connection refused to host:
localhost; nested exception is:
    java.net.ConnectException: Connection refused: connect
[12:43:31] Serveur -> +OK Password accepted
[12:43:38] /[0:0:0:0:0:0:1]:58893 -> LIST
[12:43:38] Serveur -> +OK 0 messages
[12:43:38] Serveur -> .
[12:43:48] /[0:0:0:0:0:0:1]:58893 -> RETR 1
[12:43:48] Serveur -> -ERR No such message
[12:44:07] /[0:0:0:0:0:0:1]:58893 -> QUIT
[12:44:07] Serveur -> +OK POP3 server signing off
----- Déconnexion : /[0:0:0:0:0:0:1]:58893 -----

Effacer l'historique

```

FIGURE 1.6 – Fenêtre historique POP3 – affichage coloré des échanges (cyan : client, vert : serveur, rouge : erreur)

Le serveur POP3 démarre sur le port 110. Un client se connecte et reçoit +OK POP3 server ready. Il s'authentifie avec USER mahmoud suivi de PASS 123; le serveur répond +OK User accepted puis +OK Password accepted. La commande LIST retourne +OK 0 messages (boîte vide), et RETR 1 retourne -ERR No such message, ce qui est le comportement attendu. La session se termine par QUIT avec confirmation de déconnexion.

La figure 6 montre la fenêtre d'historique détaillée avec code couleur : commandes client en cyan, réponses serveur en vert, erreurs en rouge. Cette visualisation facilite le diagnostic des sessions.

1.4 Interface du serveur IMAP

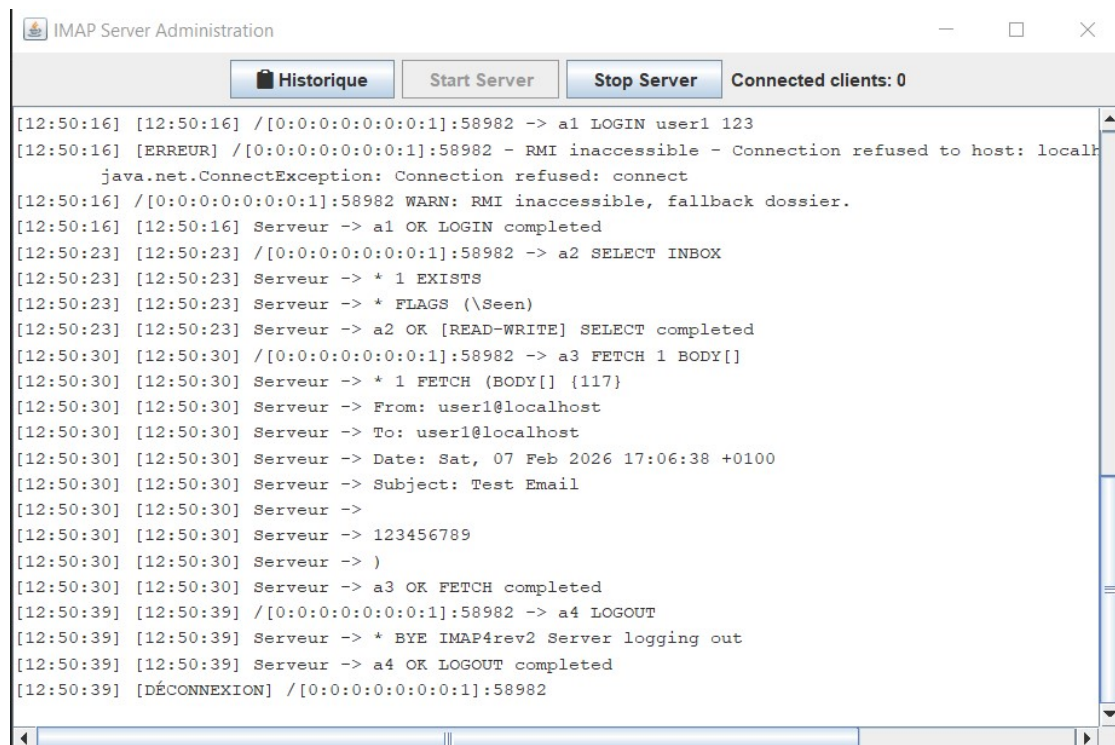


FIGURE 1.7 – Interface IMAP – session : LOGIN user1, SELECT INBOX, FETCH 1 BODY[], LOGOUT

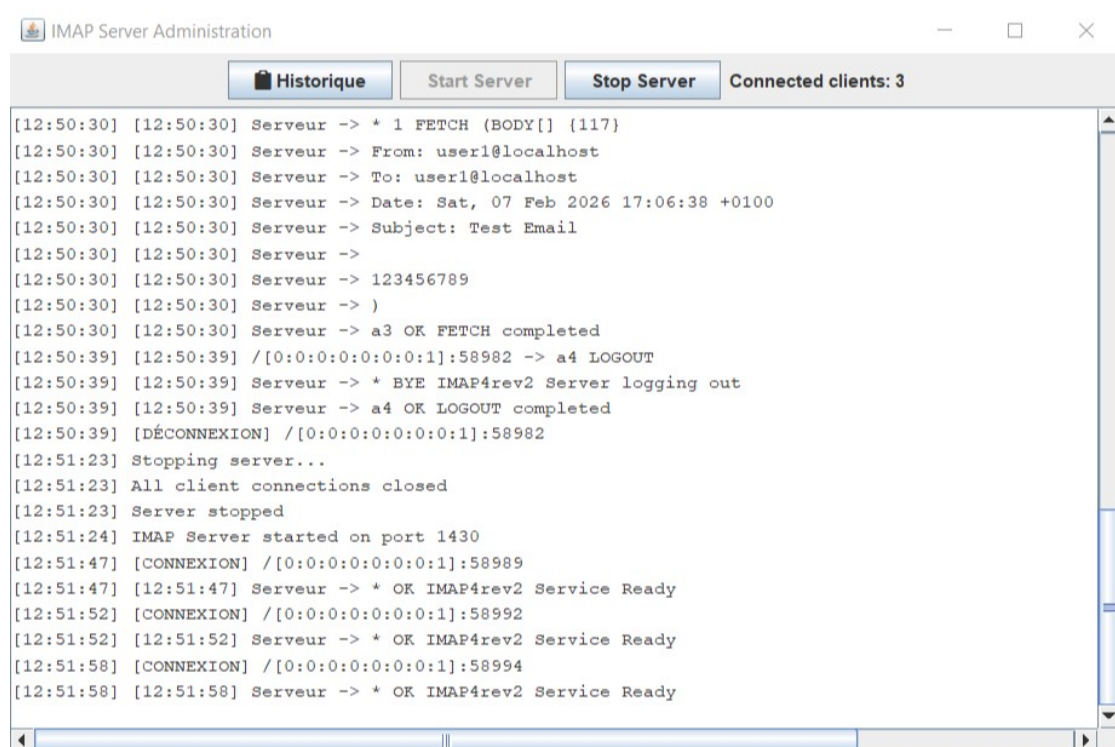


FIGURE 1.8 – Interface IMAP – redémarrage sur port 1430, trois clients connectés simultanément

La figure 7 illustre une session IMAP complète. Le client effectue `a1 LOGIN user1 123` et reçoit `a1 OK LOGIN completed`. Il sélectionne INBOX avec `a2 SELECT INBOX`; le serveur répond `* 1 EXISTS` (un message présent) et `* FLAGS (\Seen)`. La commande `a3 FETCH 1 BODY[]` retourne l'intégralité du message (en-têtes *From*, *To*, *Date*, *Subject* et corps 123456789). La session se termine par `a4 LOGOUT`.

La figure 8 montre le redémarrage du serveur sur le port 1430 et l'acceptation simultanée de 3 clients distincts (ports 58989, 58992, 58994), confirmant le bon fonctionnement du multi-threading.

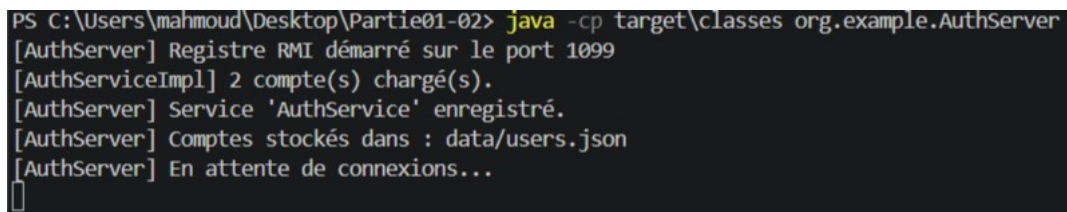
Analyse comparative IMAP / POP3 : Contrairement à POP3 qui télécharge et supprime les messages, IMAP les conserve sur le serveur et permet leur consultation partielle. L'automate à états IMAP est bien respecté : la commande `FETCH` n'est accessible qu'après `SELECT`. IMAP est donc plus adapté aux environnements multi-appareils.

Partie 2 – Serveur d’Authentification avec Java RMI

2.1 Objectif

Cette partie consiste à implémenter un serveur d’authentification distribué utilisant Java RMI. Ce serveur centralise la gestion des comptes utilisateurs et est consulté par les serveurs SMTP, POP3 et IMAP pour vérifier l’identité des utilisateurs lors de chaque connexion. Les comptes sont stockés dans un fichier `data/users.json` pour assurer la persistance entre les redémarrages.

2.2 Partie 1 – Démarrage du serveur RMI et client d’administration

A terminal window with a black background and white text. The command `java -cp target\classes org.example.AuthServer` has been executed. The output shows the RMI registry starting on port 1099, loading 2 accounts from `data/users.json`, registering the `AuthService` service, and then waiting for connections.

```
PS C:\Users\mahmoud\Desktop\Partie01-02> java -cp target\classes org.example.AuthServer
[AuthServer] Registre RMI démarré sur le port 1099
[AuthServiceImpl] 2 compte(s) chargé(s).
[AuthServer] Service 'AuthService' enregistré.
[AuthServer] Comptes stockés dans : data/users.json
[AuthServer] En attente de connexions...
█
```

FIGURE 2.1 – Démarrage du serveur `AuthServer` – registre RMI sur le port 1099, 2 comptes chargés depuis `users.json`

Le serveur RMI se lance depuis la ligne de commande. Le registre démarre sur le port 1099, l’implémentation `AuthServiceImpl` charge 2 comptes depuis `data/users.json`, enregistre le service sous le nom `AuthService` et se met en attente de connexions (*En attente de connexions...*).

A terminal window with a black background and white text. The command `java -cp target\classes org.example.AdminClient` has been entered, and the cursor is at the end of the line.

```
PS C:\Users\mahmoud\Desktop\Partie01-02> java -cp target\classes org.example.AdminClient
█
```

FIGURE 2.2 – Lancement du client d’administration `AdminClient` depuis le terminal

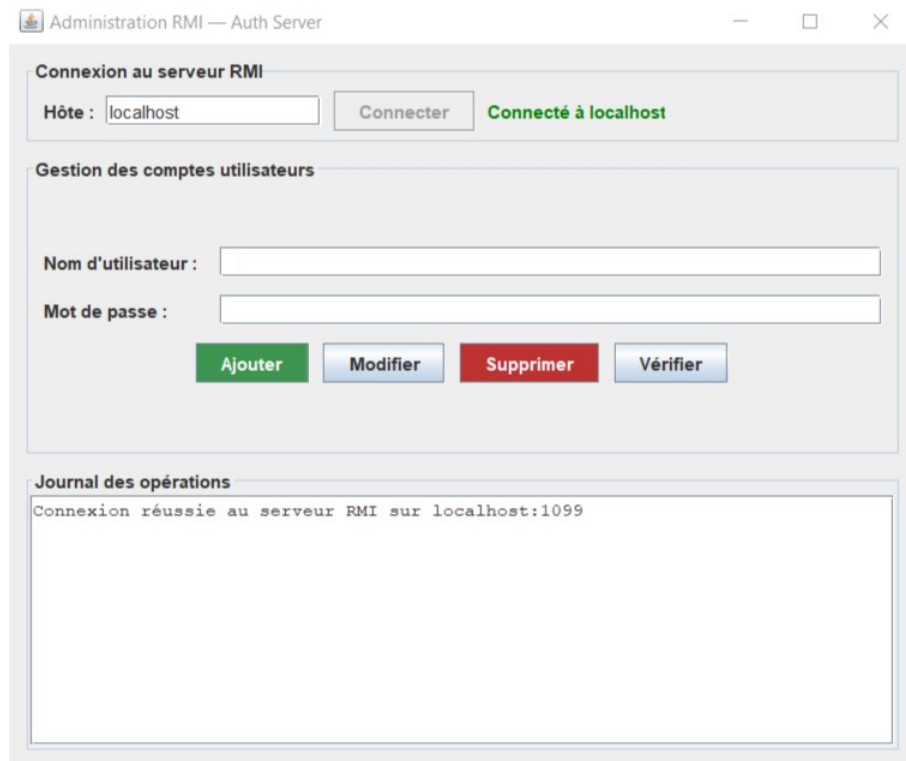


FIGURE 2.3 – Interface graphique AdminClient – connecté avec succès à localhost :1099

Les figures 10 et 11 montrent le lancement du client d’administration et son interface graphique. Le client se connecte au serveur RMI sur `localhost:1099` et confirme *Connexion réussie au serveur RMI sur localhost :1099* dans le journal d’opérations. L’interface propose quatre opérations : **Ajouter**, **Modifier**, **Supprimer** et **Vérifier** un utilisateur.

2.3 Partie 2 – Gestion des comptes utilisateurs

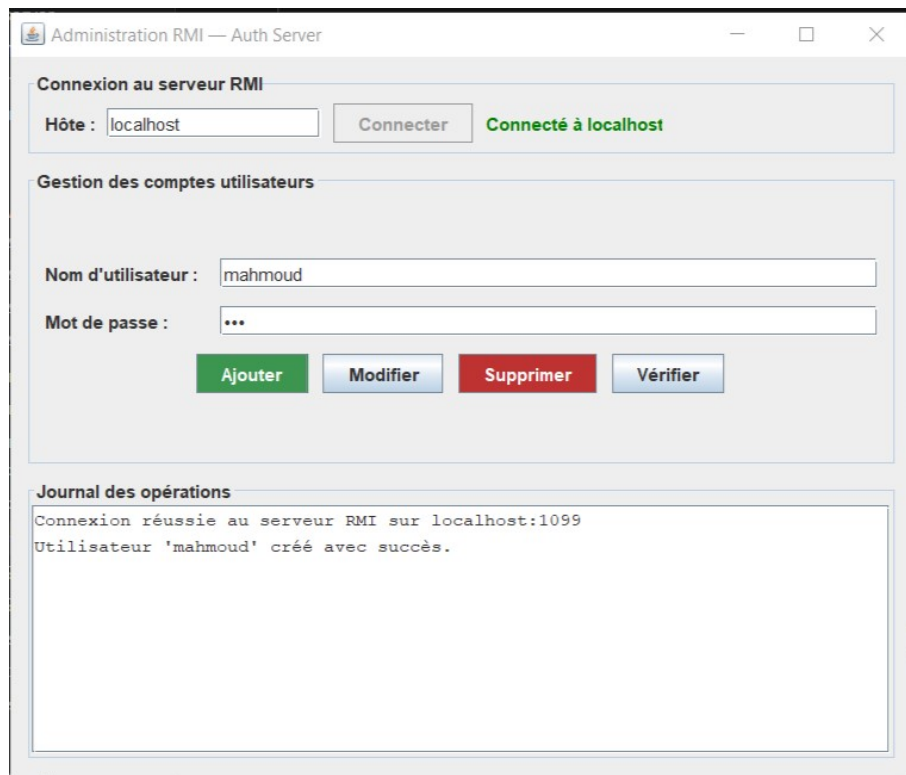


FIGURE 2.4 – Création de l'utilisateur mahmoud – journal : “Utilisateur 'mahmoud' créé avec succès”

La figure 12 montre la création réussie de l'utilisateur mahmoud. Le nom et le mot de passe sont saisis dans le formulaire, puis le bouton *Ajouter* est utilisé. Le journal confirme la création et le serveur persiste ce compte dans le fichier JSON.



FIGURE 2.5 – Confirmation côté terminal serveur – [Auth] createUser(mahmoud) -> OK

La figure 13 confirme côté serveur que l'appel RMI `createUser(mahmoud)` a retourné OK, validant la communication entre le client graphique et le service distant.

```
[Auth] createUser(mahmoud) -> OK
```

FIGURE 2.6 – Arborescence mailserver – répertoire mahmoud créé automatiquement à la création du compte

La figure 14 montre que lors de la création du compte, le serveur a automatiquement créé le répertoire `mailserver/mahmoud` pour stocker les messages de cet utilisateur. On voit également le répertoire `user1` contenant un fichier message existant (`20260207_170638.txt`), confirmant la cohérence entre les comptes RMI et le stockage des emails sur le système de fichiers.

2.4 Partie 3 – Intégration avec le serveur POP3

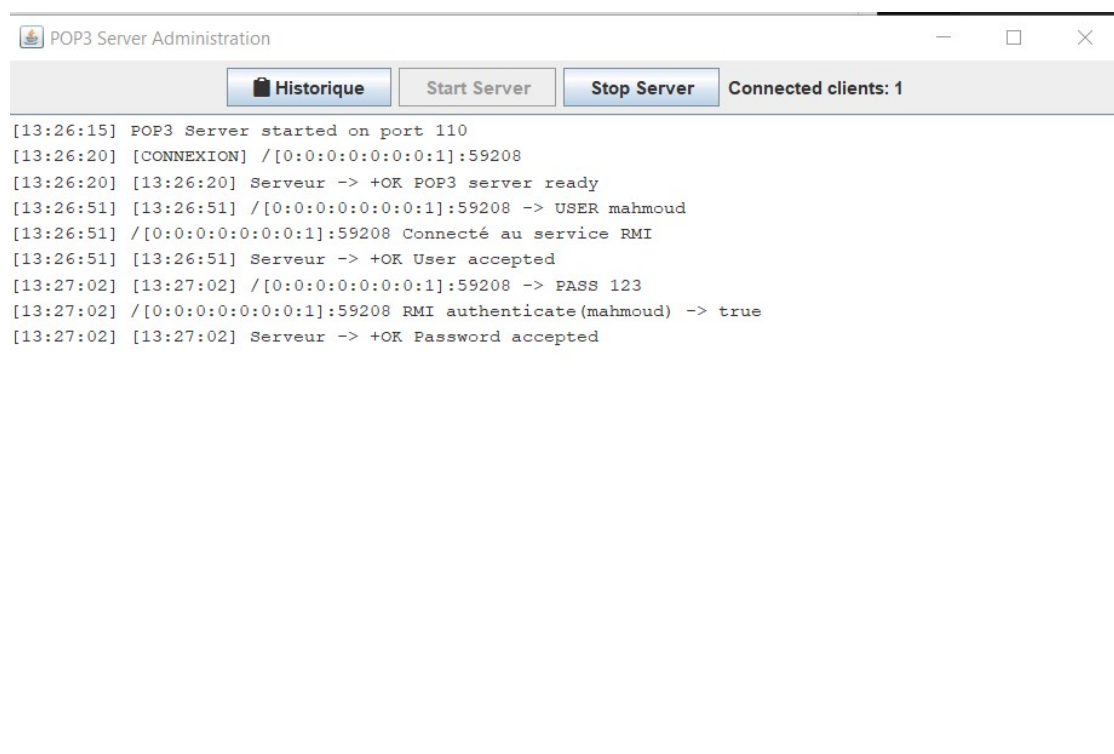
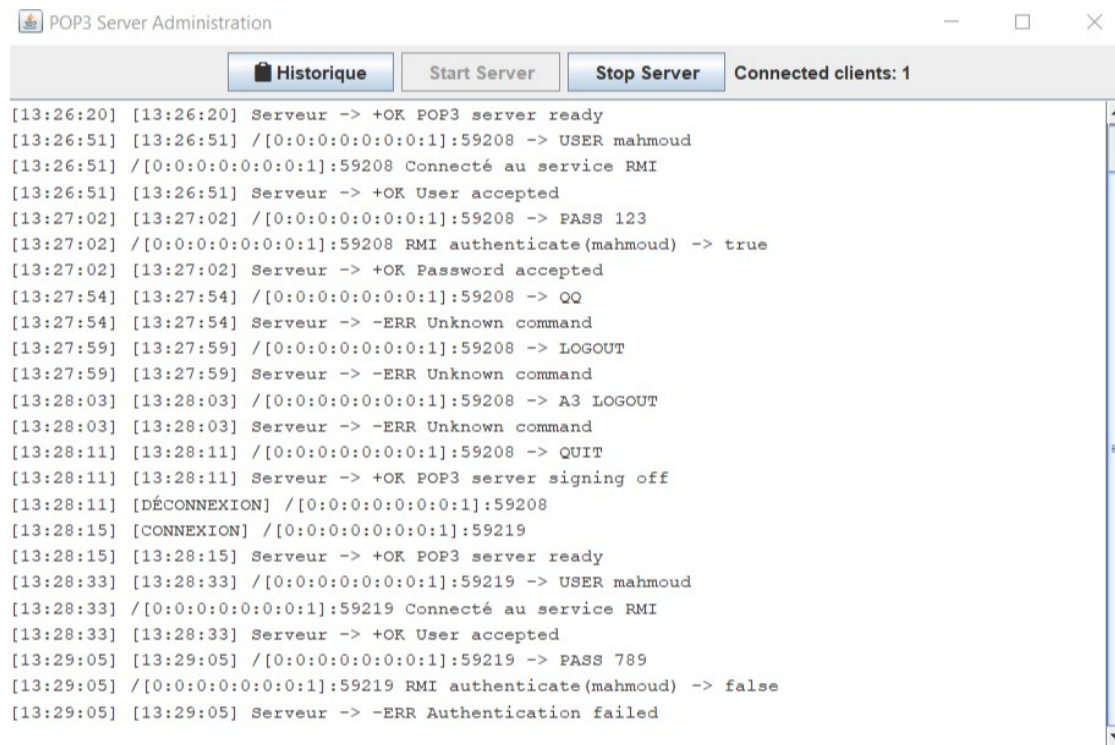


FIGURE 2.7 – Serveur POP3 avec RMI actif – RMI `authenticate(mahmoud)` -> true, mot de passe accepté

La figure 15 démontre l'intégration réussie entre le serveur POP3 et le serveur RMI. Le journal POP3 affiche *Connecté au service RMI* lors de la commande `USER mahmoud`. Lors de `PASS 123`, l'appel RMI `authenticate(mahmoud)` -> true confirme la vérification

par le service distant. Le serveur répond +OK Password accepted. Ceci contraste avec les tests de la partie 1 où le RMI était absent et la vérification se faisait localement.

2.5 Partie 4 – Tests d’authentification valide et invalide



```

[13:26:20] [13:26:20] Serveur -> +OK POP3 server ready
[13:26:51] [13:26:51] /[0:0:0:0:0:0:1]:59208 -> USER mahmoud
[13:26:51] [13:26:51] /[0:0:0:0:0:0:1]:59208 Connecté au service RMI
[13:26:51] [13:26:51] Serveur -> +OK User accepted
[13:27:02] [13:27:02] /[0:0:0:0:0:0:1]:59208 -> PASS 123
[13:27:02] [13:27:02] /[0:0:0:0:0:0:1]:59208 RMI authenticate(mahmoud) -> true
[13:27:02] [13:27:02] Serveur -> +OK Password accepted
[13:27:54] [13:27:54] /[0:0:0:0:0:0:1]:59208 -> QQ
[13:27:54] [13:27:54] Serveur -> -ERR Unknown command
[13:27:59] [13:27:59] /[0:0:0:0:0:0:1]:59208 -> LOGOUT
[13:27:59] [13:27:59] Serveur -> -ERR Unknown command
[13:28:03] [13:28:03] /[0:0:0:0:0:0:1]:59208 -> A3 LOGOUT
[13:28:03] [13:28:03] Serveur -> -ERR Unknown command
[13:28:11] [13:28:11] /[0:0:0:0:0:0:1]:59208 -> QUIT
[13:28:11] [13:28:11] Serveur -> +OK POP3 server signing off
[13:28:11] [13:28:11] [DÉCONNEXION] /[0:0:0:0:0:0:1]:59208
[13:28:15] [13:28:15] [CONNEXION] /[0:0:0:0:0:0:1]:59219
[13:28:15] [13:28:15] Serveur -> +OK POP3 server ready
[13:28:33] [13:28:33] /[0:0:0:0:0:0:1]:59219 -> USER mahmoud
[13:28:33] [13:28:33] /[0:0:0:0:0:0:1]:59219 Connecté au service RMI
[13:28:33] [13:28:33] Serveur -> +OK User accepted
[13:29:05] [13:29:05] /[0:0:0:0:0:0:1]:59219 -> PASS 789
[13:29:05] [13:29:05] /[0:0:0:0:0:0:1]:59219 RMI authenticate(mahmoud) -> false
[13:29:05] [13:29:05] Serveur -> -ERR Authentication failed

```

FIGURE 2.8 – Tests POP3 – mot de passe incorrect, commandes invalides et refus d’un compte supprimé

La figure 16 présente trois scénarios de validation :

1. **Mot de passe incorrect** : PASS 789 provoque RMI authenticate(mahmoud) -> false et -ERR Authentication failed.
2. **Commandes invalides** : QQ, LOGOUT et A3 LOGOUT sont rejetées avec -ERR Unknown command, confirmant la robustesse de l’automate à états POP3.
3. **Compte supprimé** : la connexion retourne -ERR User not found, et une commande PASS sans USER préalable retourne -ERR USER required first.

2.6 Partie 5 – Suppression d’un compte utilisateur

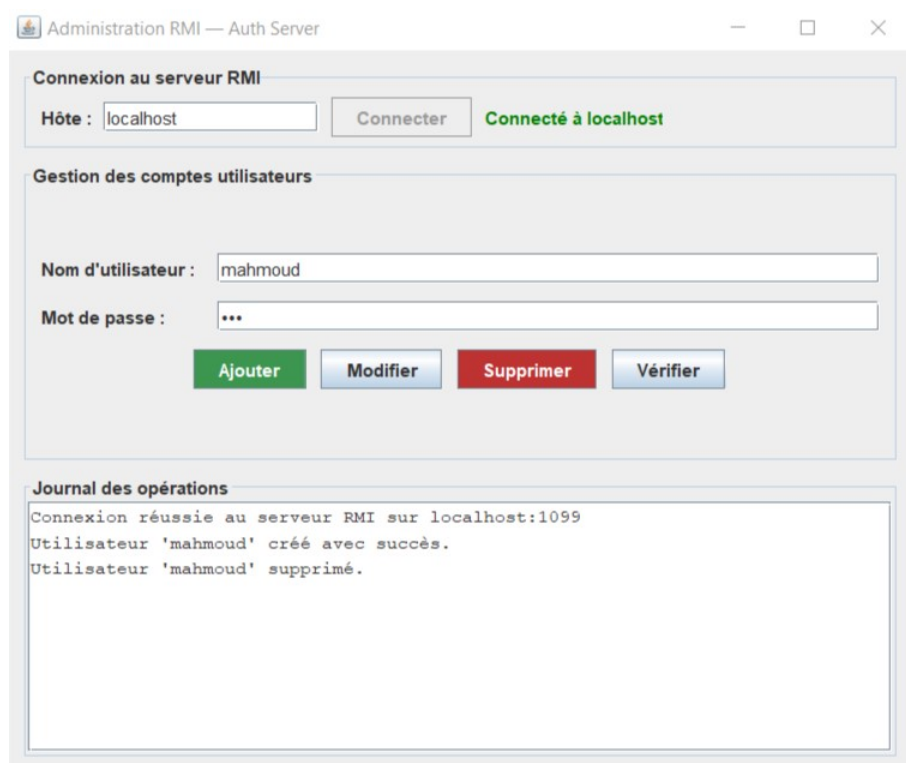


FIGURE 2.9 – Client RMI – suppression de l’utilisateur mahmoud, journal : “Utilisateur ‘mahmoud’ supprimé”

La figure 17 montre la suppression de l’utilisateur **mahmoud** via l’interface graphique. Après validation, le journal confirme *Utilisateur ‘mahmoud’ supprimé*. Cette modification est immédiatement propagée : tout serveur consultant le service RMI refusera dès lors ce compte.

2.7 Partie 6 – Modification du mot de passe

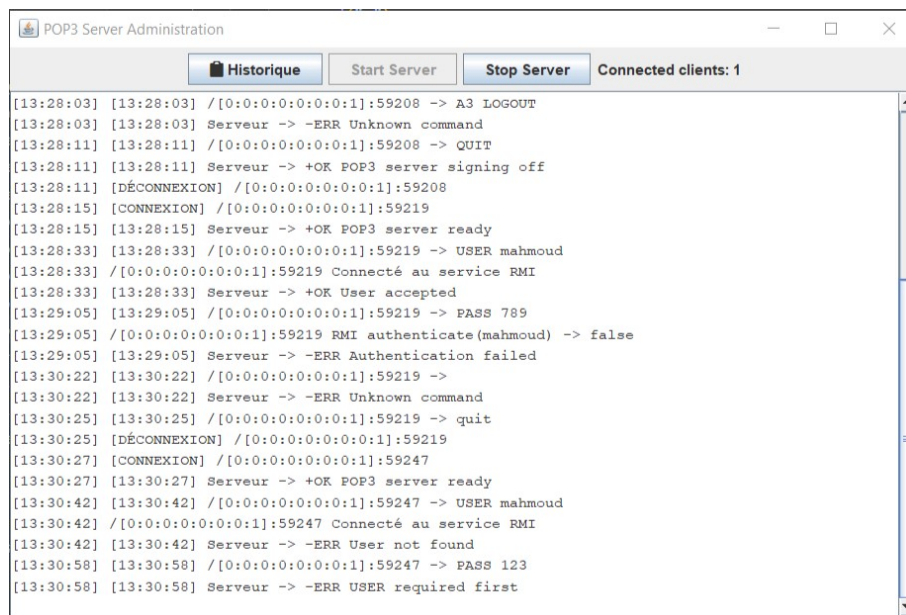


FIGURE 2.10 – Client RMI – cycle complet dans le journal : création, suppression, re-création, modification et vérifications

La figure 18 illustre le cycle complet de gestion d'un compte visible dans le journal : création, suppression, recréation, mise à jour du mot de passe (*Mot de passe de 'mahmoud' mis à jour*), puis vérification avec le bon mot de passe (*Authentification réussie pour 'mahmoud'*) et avec un mot de passe incorrect (*Mot de passe incorrect pour 'mahmoud'*). Toutes les opérations sont persistées dans le fichier JSON.

2.8 Partie 7 – Validation complète côté serveur POP3

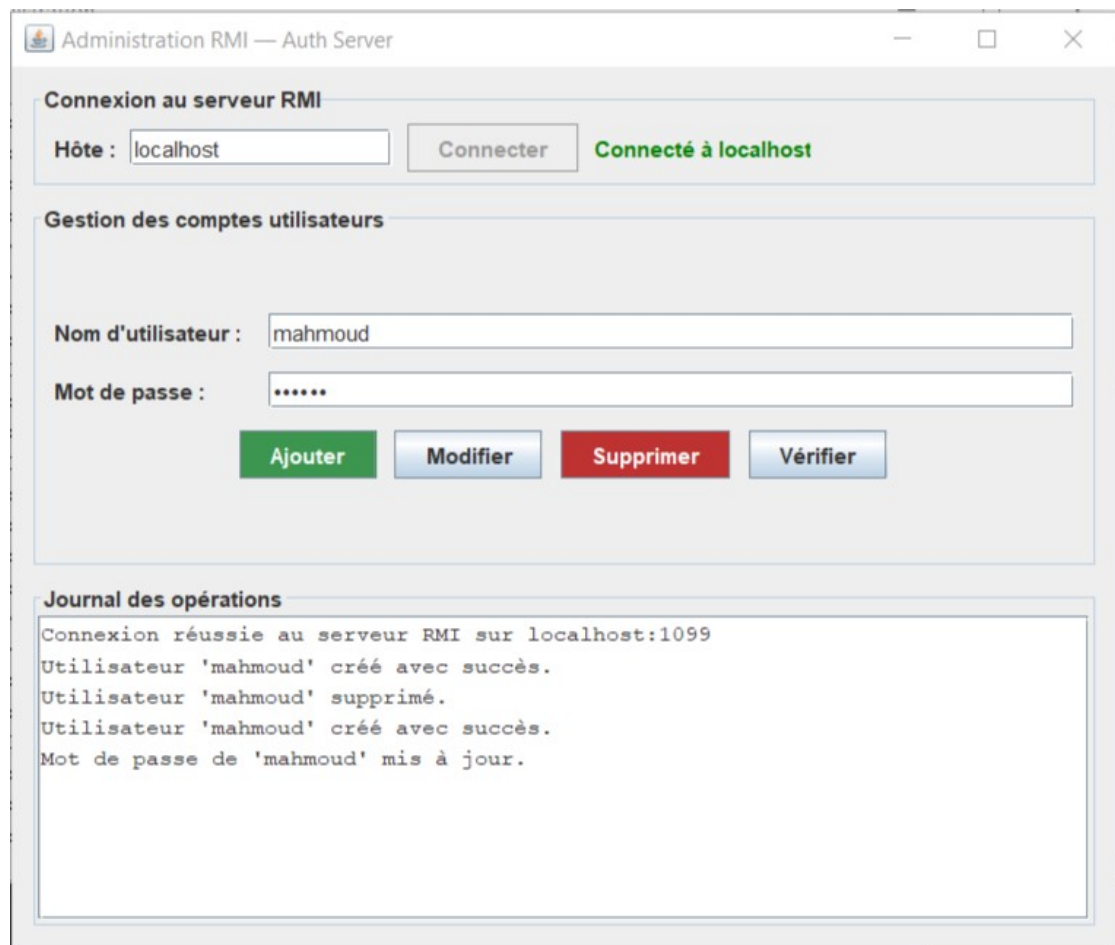


FIGURE 2.11 – Serveur POP3 – authentification via RMI : mot de passe valide (-> true) puis incorrect (-> false)

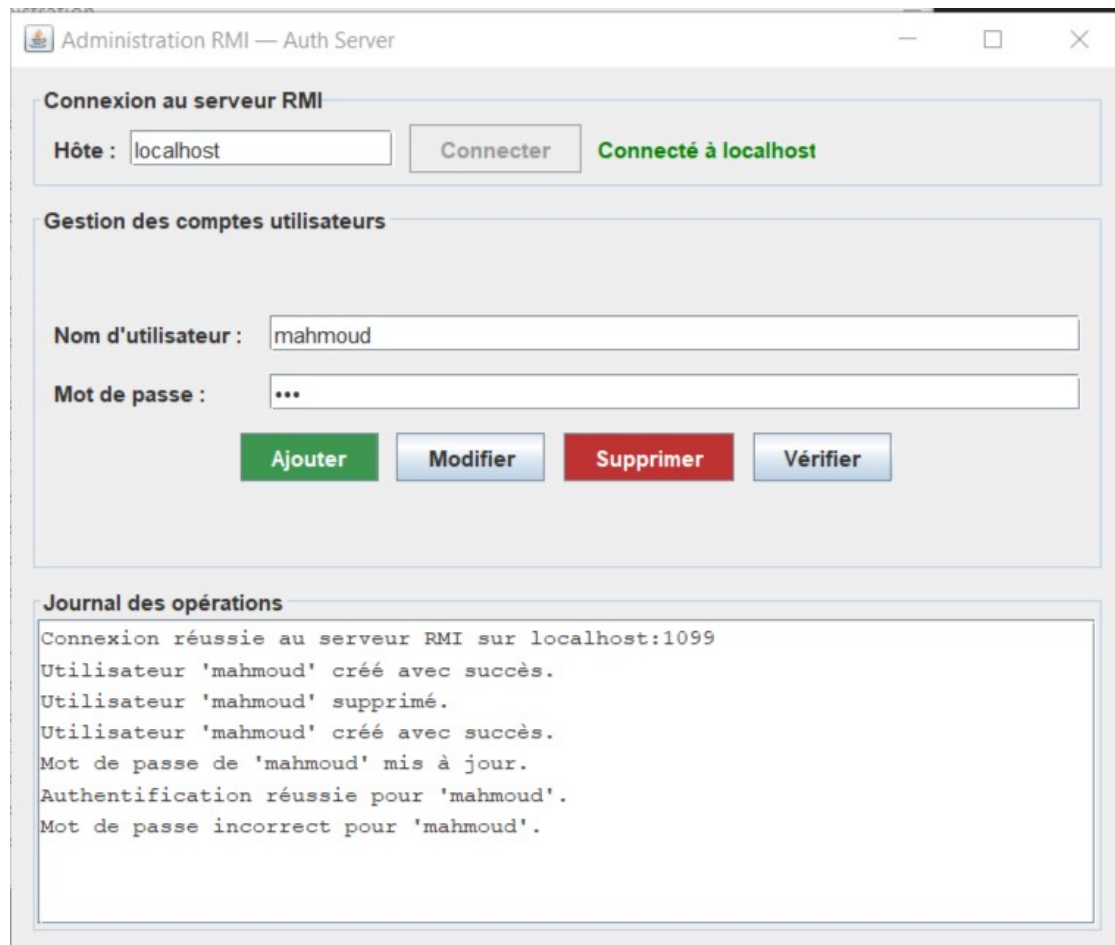


FIGURE 2.12 – Serveur POP3 – tentative de connexion avec compte supprimé (-ERR User not found)

Les figures 19 et 20 valident l'ensemble du scénario de tests demandé :

- **Identifiants valides** : `RMI authenticate(mahmoud) -> true` $\Rightarrow$ +OK Password accepted.
- **Mauvais mot de passe** : `RMI authenticate(mahmoud) -> false` $\Rightarrow$ -ERR Authentication failed.
- **Compte supprimé** : -ERR User not found lors de USER mahmoud, puis -ERR USER required first lors d'un PASS envoyé sans USER.

Analyse générale : Le serveur d'authentification RMI centralise efficacement la gestion des identités pour l'ensemble des serveurs de messagerie. L'intégration avec POP3 est fonctionnelle et tous les scénarios de test sont validés. Les améliorations possibles incluent le hachage des mots de passe (bcrypt), un mécanisme de reconnexion automatique en cas d'indisponibilité du service RMI, et la migration vers une base de données MySQL pour une meilleure scalabilité.

Conclusion

Ce travail pratique a permis d'étendre un système de messagerie distribuée existant selon deux axes principaux. Dans un premier temps, des interfaces graphiques de supervision ont été développées pour les serveurs SMTP, POP3 et IMAP, offrant une visibilité en temps réel sur les échanges et permettant un contrôle complet du cycle de vie des serveurs de manière autonome. Dans un second temps, un serveur d'authentification distribué Java RMI a été développé et intégré, centralisant la gestion des comptes utilisateurs et renforçant la cohérence du système. Les tests réalisés valident le bon fonctionnement des protocoles et leur interaction avec le service RMI. Les perspectives d'amélioration portent sur la sécurisation des mots de passe, l'intégration d'une base de données MySQL et la mise en place d'un load balancer pour la scalabilité.